End-to-End Troubleshooting of Apache2 Web Server Deployment in AWS (VPC + Ubuntu + HTTP)

Introduction

Deploying web applications in the cloud involves multiple layers such as networking, operating systems, and web servers. Even if one layer fails, the application becomes unreachable.

In this deployment setup:
- AWS VPC for network isolation
- Public Subnet for web server hosting
- Security Groups & NACL for traffic control
- Ubuntu Server
- Apache2 for HTTP web deployment

This document explains a real-time troubleshooting approach when the web server becomes inaccessible.

Architecture Overview

Internet → Internet Gateway → Route Table → Public Subnet → Security Group + NACL → Ubuntu EC2 Instance → Apache2 Web Server → Web Application

Step 1: Check EC2 Instance Status

- Ensure instance state is Running
- Status checks should be 2/2 Passed
- Start the instance if it is stopped

Step 2: Troubleshoot VPC Configuration

- Verify VPC is created properly
- Ensure CIDR range does not overlap
- Confirm Internet Gateway is attached

Common Issue: Without Internet Gateway, server cannot access internet

Step 3: Troubleshoot Subnet Configuration

Public Subnet Requirements:
- Route to Internet Gateway

- Auto assign Public IP enabled
Common Issue: Private subnet prevents internet access

Step 4: Troubleshoot Route Tables

Public Route Table must contain:
Destination: 0.0.0.0/0
Target: Internet Gateway
Common Problems:
- Wrong route table association
- Missing default route

Step 5: Troubleshoot Network ACL (NACL)

Inbound Rules:
- Allow HTTP (Port 80)
- Allow SSH (Port 22)

Outbound Rules:
- Allow All Traffic

Common Issue: Blocked HTTP traffic prevents website access

Step 6: Troubleshoot Security Groups

Required Inbound Rules:
- HTTP – Port 80
- SSH – Port 22

Common Issue: Missing HTTP rule blocks browser access

Step 7: Troubleshoot Apache2 Service

Check Apache Status:

```
sudo systemctl status apache2
```

Start Apache:

```
sudo systemctl start apache2
```

Restart Apache:

```
sudo systemctl restart apache2
```

Step 8: Check Apache Configuration

sudo apache2ctl configtest
Expected Output: Syntax OK

Step 9: Check Apache Logs

Error Logs:

```
sudo tail -f /var/log/apache2/error.log
```

Access Logs:

```
sudo tail -f /var/log/apache2/access.log
```

Step 10: Check Port Listening

```
sudo ss -tulnp | grep :80
```

Step 11: Verify Firewall (UFW)

Check Firewall:

```
sudo ufw status
```

Allow HTTP:

```
sudo ufw allow 80
```

Step 12: Check Website Files & Permissions

Default Apache Directory:

```
/var/www/html
```

Fix Permissions:

```
sudo chmod -R 755 /var/www/html
sudo chown -R www-data:www-data /var/www/html
```

Step 13: Check Disk Space

```
sudo df -h
```

Step 14: Test Server Locally

```
curl http://localhost
```

Real-Time Troubleshooting Flow

1. Check EC2 instance health
2. Verify VPC & Internet Gateway
3. Check Subnet configuration
4. Verify Route Table
5. Check NACL rules
6. Verify Security Groups
7. Check Apache2 service
8. Analyze logs

9. Verify firewall
10. Test application

Best Practices

- Monitor server resources
- Enable logging and alerts
- Keep software updated
- Maintain backup plan
- Validate configuration changes

Conclusion

Deploying Apache2 web servers in AWS involves multiple interconnected components. Failure in any layer such as VPC, subnet, route tables, NACL, security groups, or Apache configuration can bring down the application.

Following systematic troubleshooting steps helps engineers quickly identify root causes and restore services efficiently.